T3000 API & Protocol

Device Interface Deployment via BACnet — Design · built Nov 13, 2026

Device Interface Deployment via BACnet — Design

Send firmware screens from the EEZ Editor to T3000 hardware devices, and load them back for display in EEZ Studio.

1. **Push firmware to device** — Transfer the screen definitions from the editor to the hardware controller.
2. **Pull firmware from device** — Load the stored screen definitions back from the hardware for rendering in EEZ Studio.

---

1. Current State

`firmware-export.ts` transforms `.eez-project` → per-screen firmware JSON files (one per screen, ~30KB each). "Deploy" button writes files to disk — **does not reach hardware**. See EEZ Project JSON Format.

The T3000 already pushes graphic data to devices over BACnet. Firmware deploy can reuse the same path:

Existing pipe: `Browser → HTTP → Rust → FFI → C++ → BACnet → Device`

- `write_webview_data()` in `BacnetScreen.cpp:164` reads a zip file, chunks it into 200-byte blocks, and pushes it to the device via `WritePrivateData_Blocking`.
- `WEBVIEW_MESSAGE_TYPE` in `BacnetWebView.cpp:61` defines actions 0–17 — Rust has the same enum in `t3_ffi_sync_service.rs:53`.
- `call_handle_webview_msg()` in `t3_ffi_sync_service.rs:246` is the Rust → C++ bridge, calling `BacnetWebView_HandleWebViewMsg()` in `BacnetWebView_Exports.cpp:97` which dispatches to the handler.

---

2. Recommended Approach

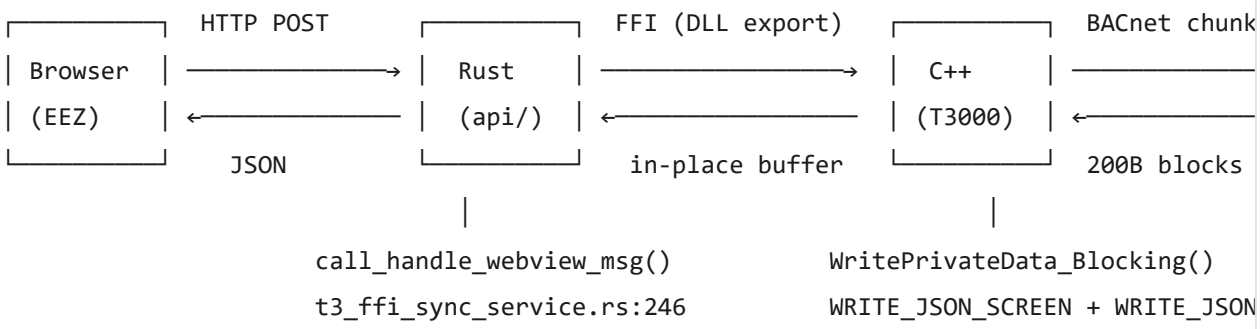
Extend the existing `WEBVIEW_MESSAGE_TYPE` enum with a new action (18) and reuse the same 200-byte chunked `WritePrivateData_Blocking` pattern from `SAVE_GRAPHIC_DATA` (action 2).

2.1. Send Firmware to Device

Layer	Change
Rust + C++ enum	Add `DEPLOY_FIRMWARE = 18` to `WebViewMessageType` in `t3_ffi_sync_service.rs:53` and `BacnetWebView.cpp:61`
Rust route	New `POST /api/devices/:id/deploy-firmware` — wraps firmware JSON with `action=18`, calls `call_handle_webview_msg()`
C++ handler	New `case DEPLOY_FIRMWARE` in `HandleWebViewMsg()` — compresses JSON, chunks 200B blocks, pushes via `WritePrivateData_Blocking(WRITE_JSON_ITEM)`
Browser	Wire "Deploy" button in `Toolbar.tsx` to the new endpoint

2.1.1. Architecture

TEXT



Why this path? The T3000 already has `SAVE_GRAPHIC_DATA` (action=2) doing exactly this — receiving graphic JSON from browser, writing to disk, zipping, chunking into 200-byte blocks, and pushing to device via `WritePrivateData_Blocking`. Firmware deploy is the same pattern: we skip the disk step and send firmware JSON directly through the Rust→C++ bridge.

2.1.2. New `WEBVIEW_MESSAGE_TYPE` Enum Extension

Both enums must stay in sync. Add to:

Rust — `api/src/t3_device/t3_ffi_sync_service.rs:53`:

RUST

```rust
pub enum WebViewMessageType {
    // ... existing 0-17 ...
    DEPLOY_FIRMWARE = 18,  // ← NEW: deploy firmware screen JSON to device
}
```

C++ — `T3000/BacnetWebView.cpp:61`:

```C++
enum WEBVIEW_MESSAGE_TYPE {
    // ... existing 0-17 ...
    DEPLOY_FIRMWARE = 18,  // ← NEW
};
```

2.1.3. JSON Message Format (Browser → Rust → C++)

Browser sends to Rust endpoint; Rust wraps with action code and forwards to C++:

```JSON
{
    "action": 18,
    "panelId": 5,
    "serialNumber": 12345,
    "screens": [
        {
            "name": "Home",
            "json": "{\"fonts\":[{\"name\":\"regular_16\",\"size\":16}],\"bitmaps\":[\"b
        }
    ]
}
```

C++ responds:

```JSON
{
    "action": "DEPLOY_FIRMWARE_RES",
    "deployed": 3,
    "failed": 0,
    "status": true
}
```

2.1.4. C++ Handler (New in `HandleWebViewMsg`)

C++

```cpp
// T3000/BacnetWebView.cpp — inside HandleWebViewMsg()

case WEBVIEW_MESSAGE_TYPE::DEPLOY_FIRMWARE:
{
    int panelId = json["panelId"].asInt();
    UINT serialNumber = json["serialNumber"].asInt();
    const Json::Value& screens = json["screens"];

    // Find BACnet device instance from panel info
    int deviceInstance = 0;
    for (auto& info : g_bacnet_panel_info) {
        if (info.nseiral_number == serialNumber && info.panel_number == panelId) {
            deviceInstance = info.object_instance; break;
        }
    }
    if (!deviceInstance) {
        WrapErrorMessage(builder, tempjson, outmsg, _T("Device not found"));
        break;
    }

    int deployed = 0, failed = 0;
    for (int i = 0; i < screens.size(); i++) {
        std::string jsonStr = screens[i]["json"].asString();
        int ret = write_firmware_raw(deviceInstance, panelId, serialNumber, i, jsonStr);
        (ret >= 0) ? deployed++ : failed++;
    }

    tempjson["action"] = "DEPLOY_FIRMWARE_RES";
    tempjson["deployed"] = deployed;
    tempjson["failed"]   = failed;
    tempjson["status"]   = (failed == 0);
    outmsg = CString(Json::writeString(builder, tempjson).c_str());
    break;
}
```

2.1.5. BACnet Transfer — New `write_firmware_raw()` (Reuses Existing Pattern)

C++

```cpp
// T3000/BacnetScreen.cpp — new function, variant of write_webview_data() line 164
// Same 200-byte chunked pattern, but accepts raw JSON string (no disk I/O)

int write_firmware_raw(int deviceInstance, int panelId,
                       UINT serialNumber, int screenIndex,
                       const std::string& jsonStr)
{
    // 1. Compress JSON in memory
    std::vector<char> compressed = ZipInMemory(jsonStr.data(), jsonStr.size());

    // 2. Verify device firmware supports the feature
    if (GetPrivateDataSaveSPBlocking(deviceInstance, READ_SETTING_COMMAND, 0, 0,
          sizeof(Str_Setting_Info), 1) <= 0) return -1;
    if (g_Device_Basic_Setting[panelId].reg.pro_info.firmware0_rev_main * 10
        + g_Device_Basic_Setting[panelId].reg.pro_info.firmware0_rev_sub < WEBVIEW_JSON_
        return -1;  // firmware too old (pre-v643)

    // 3. Set JSON flash mode flag
    g_Device_Basic_Setting[panelId].reg.webview_json_flash = 2;
    WritePrivateData_Blocking(deviceInstance, WRITE_SETTING_COMMAND, 0, 0, 5,
                              (char*)&g_Device_Basic_Setting[panelId].reg);

    // 4. Write screen metadata (zip size)
    g_json_screen_data[panelId][screenIndex].file_data.zip_size = compressed.size();
    WritePrivateData_Blocking(deviceInstance, WRITE_JSON_SCREEN,
                              screenIndex, screenIndex, 5,
                              (char*)&g_json_screen_data[panelId][screenIndex]);

    // 5. Chunk and write — 200 bytes per block, 2 blocks per BACnet write
    int totalChunks = compressed.size() / 200 + 1;
    for (int i = 0; i < totalChunks; i++) {
        int copySize = (i < totalChunks - 1) ? 200 : (compressed.size() % 200);
        memcpy(&g_json_item_data[panelId][10 * screenIndex + i],
               &compressed[i * 200], copySize);
    }

    for (int g = 0; g < totalChunks; g += BAC_READ_JSON_ITEM_GROUP_NUMBER) {
        int start = screenIndex * 10 + g;
        int end   = min(start + BAC_READ_JSON_ITEM_GROUP_NUMBER - 1, screenIndex * 10 +
        WritePrivateData_Blocking(deviceInstance, WRITE_JSON_ITEM, start, end, 5,
```

```
                                    (char*)&g_json_item_data[panelId][start]);
        if (!offline_mode) Sleep(SEND_COMMAND_DELAY_TIME);
    }
    return 0;
}
```

Constants from `T3000/global_define.h`:

- `WEBVIEW_JSON_FEATURE = 643` — minimum firmware version
- `BAC_READ_JSON_ITEM_GROUP_NUMBER = 2` — chunks per BACnet write
- `SEND_COMMAND_DELAY_TIME` — ~50ms backoff between writes
- Each screen uses 10 slots (screen × 10 to screen × 10 + 9)

2.1.6. Rust Route (New)

RUST

```rust
// NEW FILE: api/src/t3_device/firmware_deploy_routes.rs

use axum::{extract::{Path, State}, http::StatusCode, response::Json, routing::post, Rout
use serde::Deserialize;
use serde_json::{json, Value};
use crate::app_state::T3AppState;
use crate::t3_device::t3_ffi_sync_service::{call_handle_webview_msg, WebViewMessageType}

pub fn create_firmware_deploy_routes() -> Router {
    Router::new()
        .route("/api/devices/:id/deploy-firmware", post(deploy_firmware))
}

#[derive(Deserialize)]
struct DeployFirmwareRequest {
    serial_number: i32,
    screens: Vec<FirmwareScreen>,
}

#[derive(Deserialize)]
struct FirmwareScreen {
    name: String,
    json: Value,  // the full screen JSON from firmware-export.ts
}

async fn deploy_firmware(
    State(_state): State<T3AppState>,
    Path(device_id): Path<i32>,
    Json(payload): Json<DeployFirmwareRequest>,
) -> Result<Json<Value>, StatusCode> {
    // Wrap with action code, matching C++ WEBVIEW_MESSAGE_TYPE::DEPLOY_FIRMWARE = 18
    let json_str = serde_json::to_string(&json!({
        "action": WebViewMessageType::DEPLOY_FIRMWARE as i32,
        "panelId": device_id,
        "serialNumber": payload.serial_number,
        "screens": payload.screens.iter().map(|s| json!({
            "name": s.name,
            "json": s.json.to_string()
        })).collect::<Vec<_>>(),
    })).unwrap();
```

```
    // Reuse the same 65536-byte buffer pattern as all other FFI calls
    let mut buffer = vec![0u8; 65536];
    buffer[..json_str.len()].copy_from_slice(json_str.as_bytes());

    match call_handle_webview_msg(WebViewMessageType::DEPLOY_FIRMWARE as i32, &mut buffe
        Ok(0) => {
            let response: Value = serde_json::from_slice(&buffer)
                .unwrap_or(json!({"status": "ok"}));
            Ok(Json(response))
        }
        _ => Err(StatusCode::INTERNAL_SERVER_ERROR),
    }
}
```

Register in `api/src/t3_device/routes.rs`:

RUST

```
use crate::t3_device::firmware_deploy_routes::create_firmware_deploy_routes;
// inside the Router builder:
.merge(create_firmware_deploy_routes())
```

2.1.7. Browser Side (Wire Deploy Button)

TYPESCRIPT

```typescript
// In Toolbar.tsx — the "Deploy to Device" button handler
// Firmware JSON already generated by firmware-export.ts → transformToDeviceJson()

async function handleDeploy(deviceId: number, serialNumber: number) {
    const project = parseProject();  // parse .eez-project
    const deviceScreens = transformToDeviceJson(project);  // existing function
    const screens = Object.entries(deviceScreens).map(([name, screen]) => ({
        name,
        json: screen  // full DeviceScreen object
    }));

    const response = await fetch(`/api/devices/${deviceId}/deploy-firmware`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ serial_number: serialNumber, screens })
    });

    const result = await response.json();
    showNotification(`Deployed: ${result.deployed}, Failed: ${result.failed}`);
}
```

2.2. Read Firmware from Device

Gap: `GET_WEBVIEW_LIST` (action 17) reads the graphic editor format, not firmware JSON. To read firmware screens back, we need either:

- A new action (e.g., `READ_FIRMWARE = 19`) in C++ that reads firmware JSON from the device, or
- Extend the existing `Read_Webview_Data_Special()` to return firmware JSON instead of graphic format

The device firmware, on boot, parses the stored JSON and creates LVGL widgets from the definitions.

---

3. Why This Approach (Not Others)

Option	Pros	Cons	Verdict
BACnet/WS (REST)	Standard, JSON-native	Requires HTTP server on device, complex TLS	❌ Overkill for embedded
BACnet ObjectFile	Standard BACnet object	File size limits, complex state machine	❌ Poor fit for ~30KB JSON
BACnet/SC (WebSocket)	Modern transport	Requires hub, new stack, certificate mgmt	❌ Major refactor
Extend existing private transfer	Already works, proven, devs know it	Non-standard, needs new message type (18)	✅ Fastest, lowest risk

---

4. Implementation Plan

TEXT

```
Phase 1: C++ Layer (T3000_Building_Automation_System/T3000/)
  ├── BacnetWebView.cpp    — add DEPLOY_FIRMWARE = 18 to enum + case handler
  ├── BacnetScreen.cpp     — add write_firmware_raw() function
  └── BacnetScreen.h       — declare write_firmware_raw()

Phase 2: Rust Layer (api/src/t3_device/)
  ├── t3_ffi_sync_service.rs      — add DEPLOY_FIRMWARE = 18 to enum
  ├── firmware_deploy_routes.rs   — NEW: POST /api/devices/:id/deploy-firmware
  └── routes.rs                   — register create_firmware_deploy_routes()

Phase 3: Browser Layer (src/lib/t3-eez-studio/)
  ├── firmware-export.ts   — already done (transformToDeviceJson exists)
  └── Toolbar.tsx          — wire "Deploy" button to POST /api/devices/:id/deploy-firmwa

Phase 4: Device Firmware (T3000 hardware)
  ├── Receive chunks → reassemble → decompress → store to flash
  └── Parse JSON → create LVGL widgets → render UI on boot
```

---

5. Data Flow Summary

```
TEXT
```

```
Step 1: User clicks "Deploy" in EEZ Editor
   ↓  firmware-export.ts transforms .eez-project → per-screen JSON

Step 2: Browser POSTs to Rust
   ↓  POST /api/devices/5/deploy-firmware
   ↓  Body: { serial_number: 12345, screens: [{name:"Home", json:{...}}, ...] }

Step 3: Rust wraps + calls FFI
   ↓  Adds "action": 18 (DEPLOY_FIRMWARE)
   ↓  call_handle_webview_msg(18, json_buffer) → BacnetWebView_HandleWebViewMsg(18, msg,

Step 4: C++ dispatches
   ↓  HandleWebViewMsg → case DEPLOY_FIRMWARE
   ↓  Validates panelId/serialNumber → finds deviceInstance

Step 5: C++ transfers to device via BACnet
   ↓  write_firmware_raw(deviceInstance, panelId, serial#, screenIdx, jsonStr)
   ↓  Compress → chunk 200B blocks → WritePrivateData_Blocking(WRITE_JSON_ITEM)

Step 6: Device receives and stores
   ↓  Reassemble chunks → decompress → flash: /screens/Home.json
   ↓  On reload: parse JSON → create LVGL widgets → render UI
```

---

6. Device-Hosted REST API (Direct Communication)

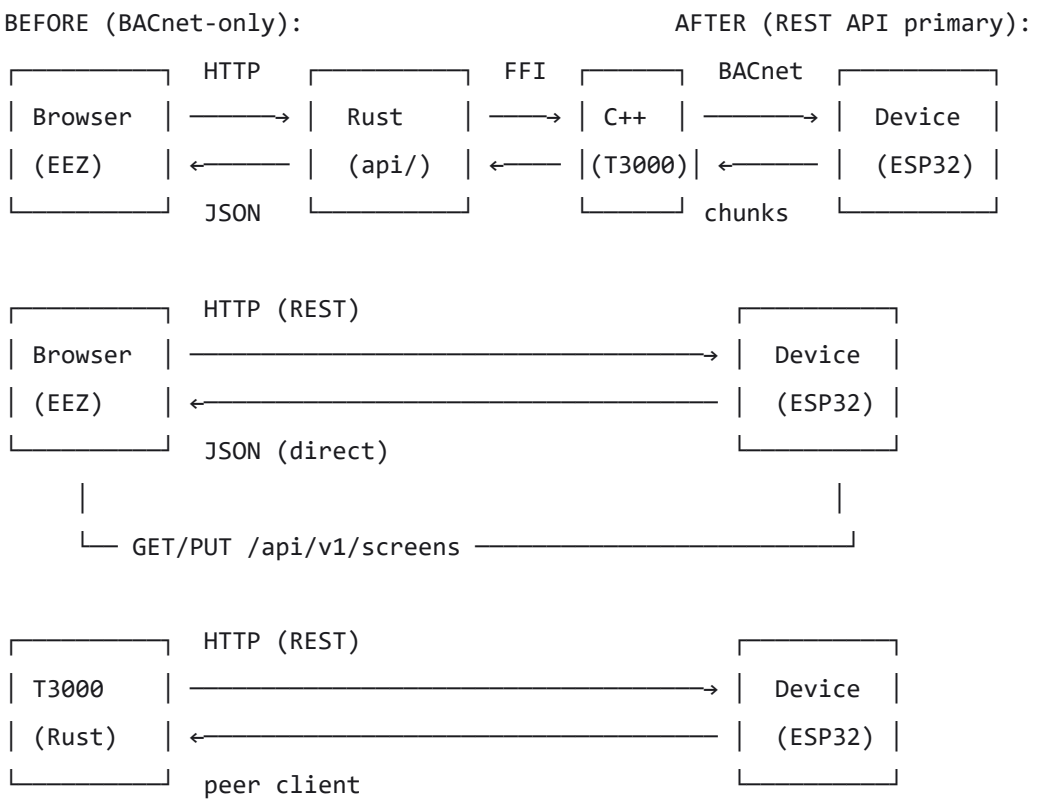
6.1. Motivation

The BACnet path routes everything through T3000 (`Browser → Rust → C++ → BACnet → Device`). The new requirement is:

- **ESP32 hosts an embedded HTTP REST API server** (e.g., ESP-IDF HTTP Server or Mongoose)
- **Clients communicate directly** with the device — no mandatory T3000 hop
- **T3000 can still read/write** device data, but as a peer HTTP client, not a mandatory gateway
- **Delta updates**: after the initial full sync, send only changed keys to reduce payload

6.2. Architecture Comparison

TEXT



6.3. Default Flow: Full Sync

The primary design is full sync — send all, load all. Delta is an optimization layered on top.

Import flow (two-step):

Step	Endpoint	Purpose
Device summary	`GET /api/v1/device/info`	Lightweight metadata: panel name, screen size, screen list, image count
Load all screens	`GET /api/v1/screens`	Pull ALL screens with full widget JSON
Deploy to device	`PUT /api/v1/screens`	Push ALL screens to device in one request

TEXT

```
First connect / Import:
  Browser ── GET /api/v1/device/info ──→ Device
  Browser ←── {panel_name, screen_size:{width,height}, screen_count,
               screens:["Home","Settings",...], image_count, ...} ── Device

  Browser ── GET /api/v1/screens ──→ Device
  Browser ←── {screens: [{name:"Home", json:{widgets:{...},fonts:[...],...}}, ...]} ── D

First deploy:
  Browser ── PUT /api/v1/screens {screens: [...]} ──→ Device
  Browser ←── {deployed: 3, status: "ok"} ── Device
```

6.4. Full API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/device/info`	**Device summary** — panel name, screen size, screen list, image/font counts (no widget JSON)
`GET`	`/api/v1/screens`	**Load all** — full JSON for every screen
`PUT`	`/api/v1/screens`	**Deploy all** — full JSON for every screen (body: `{screens: [...]}`)
`GET`	`/api/v1/screens/:name`	Load single screen
`PUT`	`/api/v1/screens/:name`	Deploy/replace single screen
`PATCH`	`/api/v1/screens/:name`	Delta update — only changed keys (optimization, see §6.5)
`PATCH`	`/api/v1/screens/:name/widgets/:id`	Delta single widget (optimization)
`POST`	`/api/v1/screens/:name/actions`	Execute action (button press, setpoint change, etc.)

6.4.1. Device Info Response (`GET /api/v1/device/info`)

JSON

```
{
    "panel_name": "T3-BB",
    "serial_number": 0,
    "screen_size": { "width": 480, "height": 320 },
    "screen_count": 11,
    "screens": [
        "start_up_screen",
        "home_screen",
        "main_menu",
        "schedule_screen",
        "schedule_edit_screen",
        "holiday_calender_screen",
        "network_config",
        "protocols",
        "parameters",
        "time",
        "wifi_config"
    ],
    "image_count": 16,
    "font_count": 1,
    "firmware_version": "5.1.0"
}
```

6.5. Delta Update Protocol (`PATCH` — Optimization)

After the initial full sync is established, subsequent updates can send only modified key paths:

TEXT

```
PATCH /api/v1/screens/Home
{
  "changes": [
    { "path": "widgets.temp_label.obj_text",           "value": "zones[2].temperature" }
    { "path": "widgets.fan_switch.state",               "value": 1 },
    { "path": "widgets.mode_btn.style.checked.bg_color", "value": 4278190080 }
  ]
}
```

The device merges these changes into its stored JSON and hot-reloads only the affected widgets — avoiding a full screen rebuild.

Response:

```JSON
{
  "applied": 3,
  "rejected": 0,
  "status": "ok"
}
```

6.6. Mixed-Mode: REST Primary + BACnet Fallback

- **Primary path**: Browser → HTTP → ESP32 REST API (direct, no T3000)
- **Fallback path**: Browser → T3000 → BACnet → Device (when device is not on the same local network, or behind a BACnet router)

The EEZ Editor detects reachability at connect time:

```TEXT
1. Try GET http://<device-ip>/api/v1/screens
   ├─ 200 OK → Use REST API (direct mode)
   └─ Timeout / Unreachable → Fall back to BACnet path through T3000
```

6.7. Device-Side Implementation Notes

ESP32 HTTP Server options:

- ESP-IDF HTTP Server (`esp_http_server.h`) — lightweight, built-in
- Mongoose — single-file embedded networking library
- ESPAsyncWebServer — popular Arduino-compatible option

Storage:

- Screens stored as individual JSON files in SPIFFS/LittleFS: `/screens/<name>.json`
- On `PUT /api/v1/screens` , each screen is written to flash and validated
- On boot, device reads all screen JSONs and builds LVGL widgets

Concurrency:

- The HTTP server runs on a dedicated FreeRTOS task
- Screen updates are queued; the LVGL task applies them on the next render cycle
- Mutex guards the shared screen JSON state

6.8. Data Flow Summary (REST API Path)

```
TEXT
```

```
Step 1: User clicks "Deploy" in EEZ Editor
    → firmware-export.ts transforms .eez-project → per-screen JSON

Step 2: Browser sends directly to device
    → PUT http://<device-ip>/api/v1/screens
    → Body: { screens: [{name:"Home", json:{...}}, ...] }

Step 3: ESP32 HTTP server receives
    → Validates JSON structure
    → Writes each screen to flash: /screens/<name>.json

Step 4: ESP32 applies changes
    → Queues screen update for LVGL task
    → LVGL task rebuilds affected widgets on next render cycle

Step 5: Device responds
    → { deployed: 3, failed: 0, status: "ok" }
```

6.9. Image/Icon API — Binary Assets

Images and icons (bitmaps, PNGs, compressed LVGL image formats) are binary assets, not JSON. They are transferred as complete files — never partial or delta updates. The display JSON references them by `imageId` rather than embedding raw bytes.

6.9.1. Design Rationale

Concern	Decision
Embed vs. reference	Reference by `imageId` — keeps display JSON lean (~30KB), avoids base64 bloat
Transfer unit	Always full file — images are opaque binary blobs; partial updates are meaningless
Caching	`ETag` + `If-None-Match` — device returns 304 if the image hasn't changed
Format	LVGL-compatible binary (`.bin` header + RGBA/INDEXED pixel data), or raw PNG for editor convenience
Storage	Per-image file in SPIFFS/LittleFS: `/images/<id>.bin`

6.9.2. API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/images`	**List all** — returns `{ images: [{id, name, width, height, format, size, checksum}] }`
`GET`	`/api/v1/images/:id`	**Download** — binary image blob (Content-Type: `application/octet-stream`)
`GET`	`/api/v1/images/:id/info`	**Metadata** — `{id, name, width, height, format, size, checksum}` without the binary body
`PUT`	`/api/v1/images/:id`	**Upload/replace** — binary body, creates or overwrites
`POST`	`/api/v1/images`	**Upload new** — multipart form with `id`, `name`, and binary `file` field; returns `{id, status}`
`DELETE`	`/api/v1/images/:id`	**Remove** — delete image from device flash
`GET`	`/api/v1/images/:id?format=png`	**Convert & download** — device converts `.bin` → PNG on-the-fly for editor previews

6.9.3. How Display JSON References Images

Screen JSON uses an `imageId` field instead of embedding pixel data:

JSON

```
{
  "name": "Home",
  "bitmaps": [
    { "id": "bg_home",      "name": "bg_home.bin",      "type": "background" },
    { "id": "icon_fan_on",  "name": "icon_fan_on.bin",  "type": "icon" },
    { "id": "icon_fan_off", "name": "icon_fan_off.bin", "type": "icon" }
  ],
  "widgets": {
    "bg": {
      "type": "Image",
      "imageId": "bg_home",
      "x_pos": 0, "y_pos": 0,
      "width": 480, "height": 320
    },
    "fan_icon": {
      "type": "Image",
      "imageId": "icon_fan_off",
      "action": "setImageId",
      "bindings": [
        { "property": "imageId", "expression": "fans[0].running ? 'icon_fan_on' : 'icon_
      ]
    }
  }
}
```

Key rules:

- `bitmaps` declares all images the screen needs — the editor uses this for bundle calculation.
- `imageId` in a widget is a string key, resolved to a file via the bitmaps table.
- Dynamic image switching (e.g., fan on/off) uses `imageId` binding expressions — the device maps the string to the cached binary at render time.
- The `imageId` namespace is **per-device** (not per-screen). Screens can share images to avoid duplicate transfers.

6.9.4. Upload Flow

TEXT

```
Step 1: Editor collects all imageIds referenced by the project
    → scans .eez-project bitmaps → resolves to local file paths

Step 2: Check what the device already has
    → GET /api/v1/images
    → Response: { images: [{id:"bg_home", checksum:"a1b2c3"}, ...] }

Step 3: Compute diff — skip images with matching checksums
    → only upload new or changed images

Step 4: Upload each image (parallel for throughput, sequential for embedded devices)
    → PUT /api/v1/images/bg_home
    → Header: Content-Type: application/octet-stream
    → Header: X-Image-Name: bg_home.bin
    → Header: X-Image-Checksum: sha256:a1b2c3d4...
    → Body: <binary pixel data>

Step 5: Device stores
    → Validates binary header (magic bytes, dimensions, format)
    → Writes to flash: /images/bg_home.bin
    → Updates checksum registry for fast diff on next connect
    → Responds: { id: "bg_home", status: "stored", checksum: "sha256:a1b2c3d4..." }
```

6.9.5. Download Flow (Editor: Import from Device)

TEXT

```
Step 1: Editor requests image list
    → GET /api/v1/images
    → Knows which images exist, their sizes, and checksums

Step 2: For each image needed by the editor:
    → GET /api/v1/images/bg_home
    → Header: If-None-Match: "a1b2c3d4"   ← client's cached checksum

Step 3: Device responds:
    → 200 OK + binary body          (image is new or changed)
    → 304 Not Modified              (client already has this version — skip)
```

6.9.6. Caching & ETag

The device computes an ETag from the image's content hash (SHA-256 or CRC32 for constrained devices):

```
TEXT
```

```
GET /api/v1/images/bg_home
→ 200 OK
  ETag: "a1b2c3d4e5f6..."
  Content-Type: application/octet-stream
  Content-Length: 153600
  X-Image-Format: rgba8888
  X-Image-Width: 480
  X-Image-Height: 320

GET /api/v1/images/bg_home
  If-None-Match: "a1b2c3d4e5f6..."
→ 304 Not Modified
```

This avoids retransferring large images that haven't changed between editing sessions.

6.9.7. Format Negotiation

The device stores images in its native LVGL binary format. The editor can request conversion:

Query	Response Content-Type	Use Case
(none)	`application/octet-stream`	Raw LVGL `.bin` for display rendering
`?format=png`	`image/png`	Editor preview, thumbnail generation
`?format=raw`	`application/octet-stream` + headers	Machine-to-machine transfer

If the device cannot convert (`?format=png` on a memory-constrained MCU), it returns `501 Not Implemented`. The editor falls back to downloading the raw `.bin` and converting client-side (via `lv_img_conv` in the browser).

6.9.8. Device-Side Implementation Notes

Storage layout:

TEXT

```
/spiffs/images/
├── _registry.json          ← { "bg_home": {"file":"bg_home.bin","checksum":"a1b2",...},
├── bg_home.bin
├── icon_fan_on.bin
└── icon_fan_off.bin
```

ESP32 handler pseudocode:

C

```c
// GET /api/v1/images/:id
esp_err_t image_get_handler(httpd_req_t *req) {
    char id[64];
    get_path_param(req, "id", id);

    // Check ETag for 304
    char etag[65];
    registry_get_checksum(id, etag);
    if (check_if_none_match(req, etag)) {
        httpd_resp_set_status(req, "304 Not Modified");
        return httpd_resp_send(req, NULL, 0);
    }

    // Read from flash
    char path[128];
    snprintf(path, sizeof(path), "/spiffs/images/%s.bin", id);
    FILE *f = fopen(path, "rb");
    if (!f) { httpd_resp_send_404(req); return ESP_FAIL; }

    // Stream binary response
    httpd_resp_set_type(req, "application/octet-stream");
    httpd_resp_set_hdr(req, "ETag", etag);
    httpd_resp_set_hdr(req, "X-Image-Format", "rgba8888");

    char buf[1024];
    size_t n;
    while ((n = fread(buf, 1, sizeof(buf), f)) > 0) {
        httpd_resp_send_chunk(req, buf, n);
    }
    httpd_resp_send_chunk(req, NULL, 0);  // end chunked
    fclose(f);
    return ESP_OK;
}

// PUT /api/v1/images/:id
esp_err_t image_put_handler(httpd_req_t *req) {
    char id[64];
    get_path_param(req, "id", id);

    // Read body in chunks, write to flash
```

```
    char path[128];
    snprintf(path, sizeof(path), "/spiffs/images/%s.bin", id);
    FILE *f = fopen(path, "wb");

    char buf[1024];
    int received, total = 0;
    while ((received = httpd_req_recv(req, buf, sizeof(buf))) > 0) {
        fwrite(buf, 1, received, f);
        total += received;
    }
    fclose(f);

    // Update registry
    char checksum[65];
    compute_sha256(path, checksum);
    registry_set(id, path, total, checksum);

    // Respond
    char resp[256];
    snprintf(resp, sizeof(resp), "{\"id\":\"%s\",\"status\":\"stored\",\"size\":%d,\"che
    httpd_resp_set_type(req, "application/json");
    return httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
}
```

Memory budget (ESP32):

- 1024-byte streaming buffer (reused for both GET and PUT)
- Registry in RAM: ~100 bytes per image × typical 20 images = 2KB
- Flash budget: depends on SPIFFS partition size — typically 1–4MB for images
- No image decoding in the HTTP handler — LVGL decodes from flash on render demand

6.9.9. Updated Full API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/screens`	Load all screens
`PUT`	`/api/v1/screens`	Deploy all screens
`GET`	`/api/v1/screens/:name`	Load single screen
`PUT`	`/api/v1/screens/:name`	Deploy/replace single screen
`PATCH`	`/api/v1/screens/:name`	Delta update — only changed keys
`PATCH`	`/api/v1/screens/:name/widgets/:id`	Delta single widget
`POST`	`/api/v1/screens/:name/actions`	Execute action
`GET`	`/api/v1/images`	**List all images**
`GET`	`/api/v1/images/:id`	**Download image binary**
`GET`	`/api/v1/images/:id/info`	**Image metadata**
`PUT`	`/api/v1/images/:id`	**Upload/replace image**
`POST`	`/api/v1/images`	**Upload new image (multipart)**
`DELETE`	`/api/v1/images/:id`	**Remove image**

6.9.10. Full Deploy Flow (Screens + Images)

```
User clicks "Deploy":

Phase 1 — Images first (larger, fewer round-trips)
  ├── GET  /api/v1/images                    → list + checksums
  ├── Diff: skip matching checksums
  └── PUT  /api/v1/images/bg_home            → for each new/changed image
      PUT  /api/v1/images/icon_fan_on
      PUT  /api/v1/images/icon_fan_off
      PUT  /api/v1/images/icon_mode_heat

Phase 2 — Screens (now safe: all imageIds resolve)
  └── PUT  /api/v1/screens                   → full JSON deployment

Result: Device has all images on flash before any screen references them.
```